Contents

Get Started with gRPC in IntelliJ with Spring Boot/Maven 1

Step 3.1 – Get the Spring Boot Maven Project 1

Step 3.2 Define the service contract 2

3.2.1 Create a proto folder in src\main folder 2

Step 3.3 Generate Code based on the service contract 2

3.3.1 Generate gRPC stubs with mvn compile (or mvn package) 2

Step 3.4 Implement the Server extending the generated stubs 3

3.4.1 Implement the gRPC Service the Spring way 3

3.4.2 Run and test the server with BloomRPC 4

Step 3.5 – Implement the Client 5

In this set of exercises, we'll practice implementing client-server Spring Boot application that communicates through gRPC using IntelliJ and Maven. The contract is already defined defined using protocol buffers.

Get Started with gRPC in IntelliJ with Spring Boot/Maven

The service to be implemented will have two RPC methods to simulate the Vinyl library from the previous semester.

Step 3.1 – Get the Spring Boot Maven Project

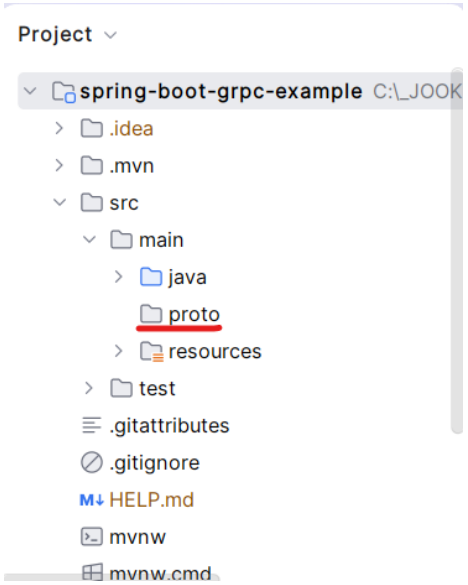
If not already done, download or clone the project from GitHub using the following link:

https://github.com/viajook/spring-boot-grpc-server-example.git

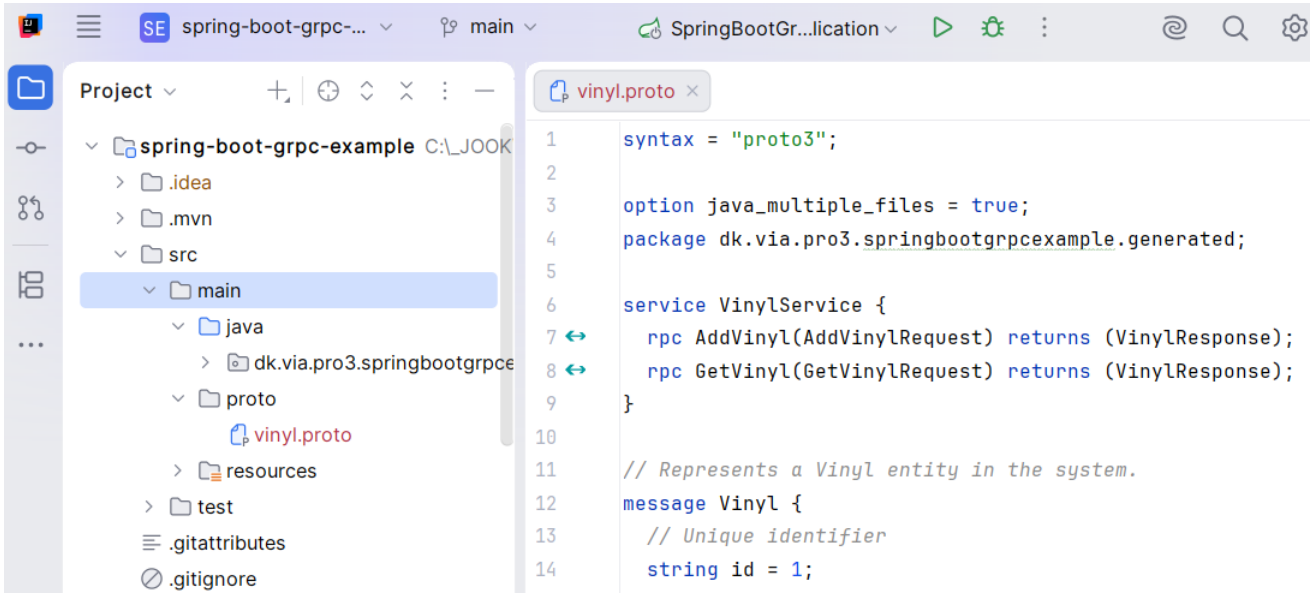
Step 3.2 Define the service contract

3.2.1 Create a proto folder in src\main folder

If not already there, create a proto folder to store the .proto files.



Your gRPC API lives in a .proto file. Here's an example from vinyl.proto:



We'll want to define the stubs for a Java service based on the contract (with ./mvnw clean compile)

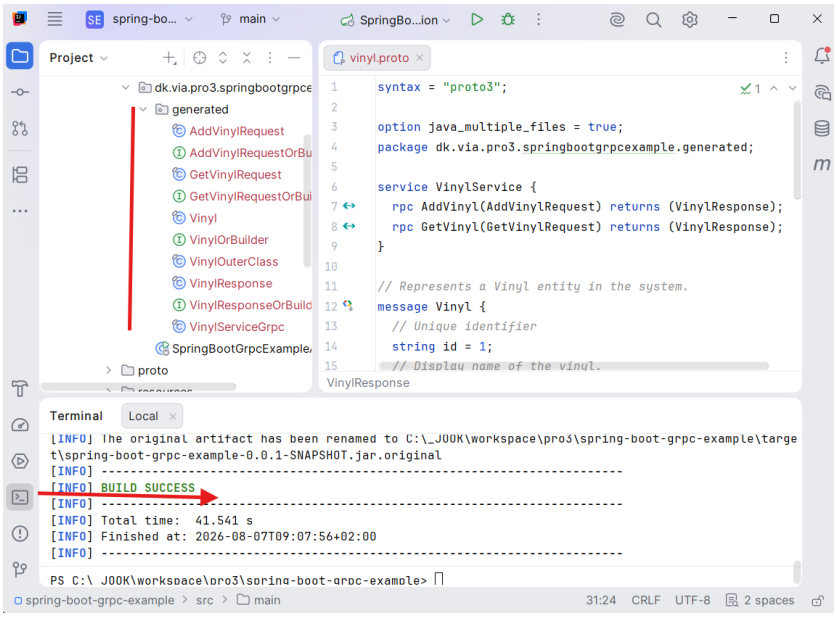
Step 3.3 Generate Code based on the service contract

3.3.1 Generate gRPC stubs with mvn compile (or mvn package)

Open Maven Tool Window on the right side of the IntelliJ, navigate to package (or compile) and click on the run button. This needs to be done every time you make changes to the .proto file. You can also use the command line as shown below:

N/B: Observe the project structure before running the command (and observe the logs during execution)

On successful execution, something like the following will be displayed:



Step 3.4 Implement the Server extending the generated stubs

3.4.1 Implement the gRPC Service the Spring way

Now we can implement the service based on the generated stubs. The magic happens with @GrpcService (and @Service). Create a service package in src\main\java\dk\via\pro3\springbootgrpcexample\service and (model)

Add the following code:

Model:

```
package dk.via.pro3.springbootgrpcexample.model;

public record VinylRecord(String id, String name) {
}
```

Service:

```
@Service
public class VinylService {
   private final Map<String, VinylRecord> vinylMap = new ConcurrentHashMap<>();

   // add vinyl record
   public Vinyl addVinyl(String name) {
      var id = UUID.randomUUID().toString();
      var vinyl = new Vinyl(id, name);
      vinylMap.put(id, vinyl);
      return vinyl;
   }
```

```
    // get vinyl record by id
    public Vinyl getVinylById(String id) {
        return vinylMap.get(id);

    }
}
```

Server:

```
@GrpcService
public class GrpcVinylServer extends VinylServiceGrpc.VinylServiceImplBase {
    private static final Logger LOG = LoggerFactory.getLogger(GrpcVinylServer.class.getName());

    private final VinylService vinylService;

    public GrpcVinylServer(VinylService service) {
        this.vinylService = service;
    }

    @Override
    public void addVinyl(AddVinylRequest request, StreamObserver<VinylResponse> responseObserver) {
        //super.addVinyl(request, responseObserver);
        var vinyl = vinylService.addVinyl(request.getName());

        var response = VinylResponse.newBuilder()
                .setVinyl(VinylMapper.toProto(vinyl))
                .build();
        responseObserver.onNext(response);
        LOG.info("\nVinyl record [ {} ] added successfully\n", vinyl);
        responseObserver.onCompleted();
    }

    @Override
    public void getVinyl(GetVinylRequest request, StreamObserver<VinylResponse> responseObserver) {
        super.getVinyl(request, responseObserver);
        // complete this part
    }

    private static class VinylMapper {
        static Vinyl toProto(VinylRecord vinylRecord) {
            return Vinyl.newBuilder()
                    .setId(vinylRecord.id())
                    .setName(vinylRecord.name())
                    .build();
        }
    }
}
```

3.4.2 Run and test the server with BloomRPC

Now we can run the program in the usual way. Start the server and test the service:

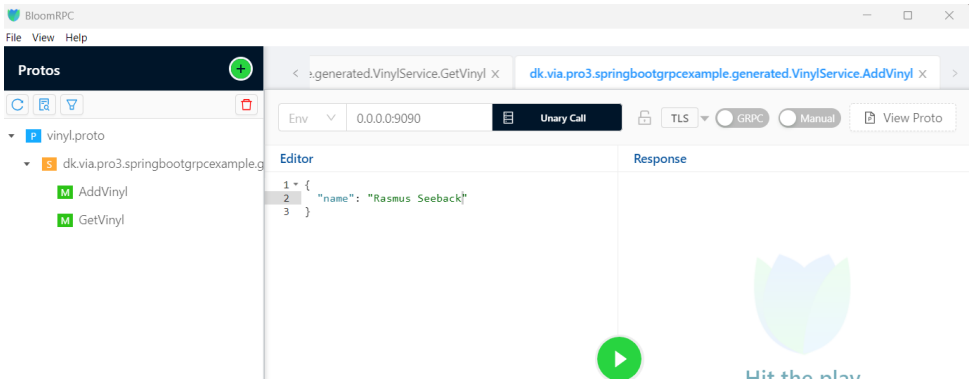
.\mvnw spring-boot:run

```
Terminal   Local                                                    AI Agents
 :: Spring Boot ::                (v4.1.0)

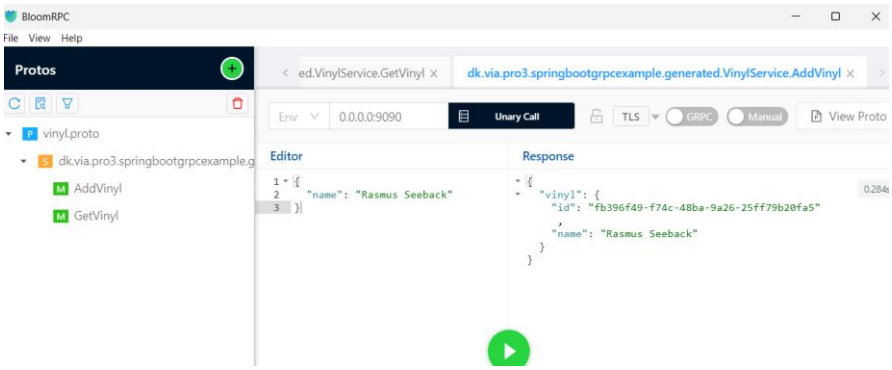
2026-08-07T10:36:45.079+02:00  INFO 19692 --- [spring-boot-grpc-example] [           main] d.v.p.s.SpringBootGrp
cExampleApplication : Starting SpringBootGrpcExampleApplication using Java 25.0.3 with PID 19692 (C:\_JOOK\works
pace\pro3\spring-boot-grpc-example\target\classes started by JOOK in C:\_JOOK\workspace\pro3\spring-boot-grpc-ex
ample)
2026-08-07T10:36:45.081+02:00  INFO 19692 --- [spring-boot-grpc-example] [           main] d.v.p.s.SpringBootGrp
cExampleApplication : No active profile set, falling back to 1 default profile: "default"
2026-08-07T10:36:45.550+02:00  INFO 19692 --- [spring-boot-grpc-example] [           main] o.s.grpc.server.Netty
GrpcServerFactory   : Registered gRPC service: dk.via.pro3.springbootgrpcexample.generated.VinylService
2026-08-07T10:36:45.551+02:00  INFO 19692 --- [spring-boot-grpc-example] [           main] o.s.grpc.server.Netty
GrpcServerFactory   : Registered gRPC service: grpc.reflection.v1.ServerReflection
2026-08-07T10:36:45.551+02:00  INFO 19692 --- [spring-boot-grpc-example] [           main] o.s.grpc.server.Netty
GrpcServerFactory   : Registered gRPC service: grpc.health.v1.Health
2026-08-07T10:36:45.671+02:00  INFO 19692 --- [spring-boot-grpc-example] [           main] o.s.g.s.lifecycle.Grp
cServerLifecycle    : gRPC Server started, listening on address: [/[0:0:0:0:0:0:0:0]:9090]
2026-08-07T10:36:45.673+02:00  INFO 19692 --- [spring-boot-grpc-example] [           main] d.v.p.s.SpringBootGrp
cExampleApplication : Started SpringBootGrpcExampleApplication in 0.79 seconds (process running for 1.021)
```

Test with BloomRPC

\+ > locate proto > click on the service > enter in the editor



Click on the Play button.



Ctrl + C, to stop the server

Step 3.5 – Implement the Client

Create a new Spring Boot project for the client and implement the client service similar to above but with gRPC Client dependency. This is left as an exercise …